



► **SENIOR RESEARCH LAB PRESENTATION: JMESSAGE**

Sam Stankiewicz & Nidhish Shakya
Computer Systems Lab, Yilmaz, Pd. 3,
5/20/2026

► PROBLEM



AMBIGUITY

Communicating with other students is bothersome without immediate directory of classmates.



PHONES

Phones, the default medium of communication, are banned. This leaves a need for a new method of communication.

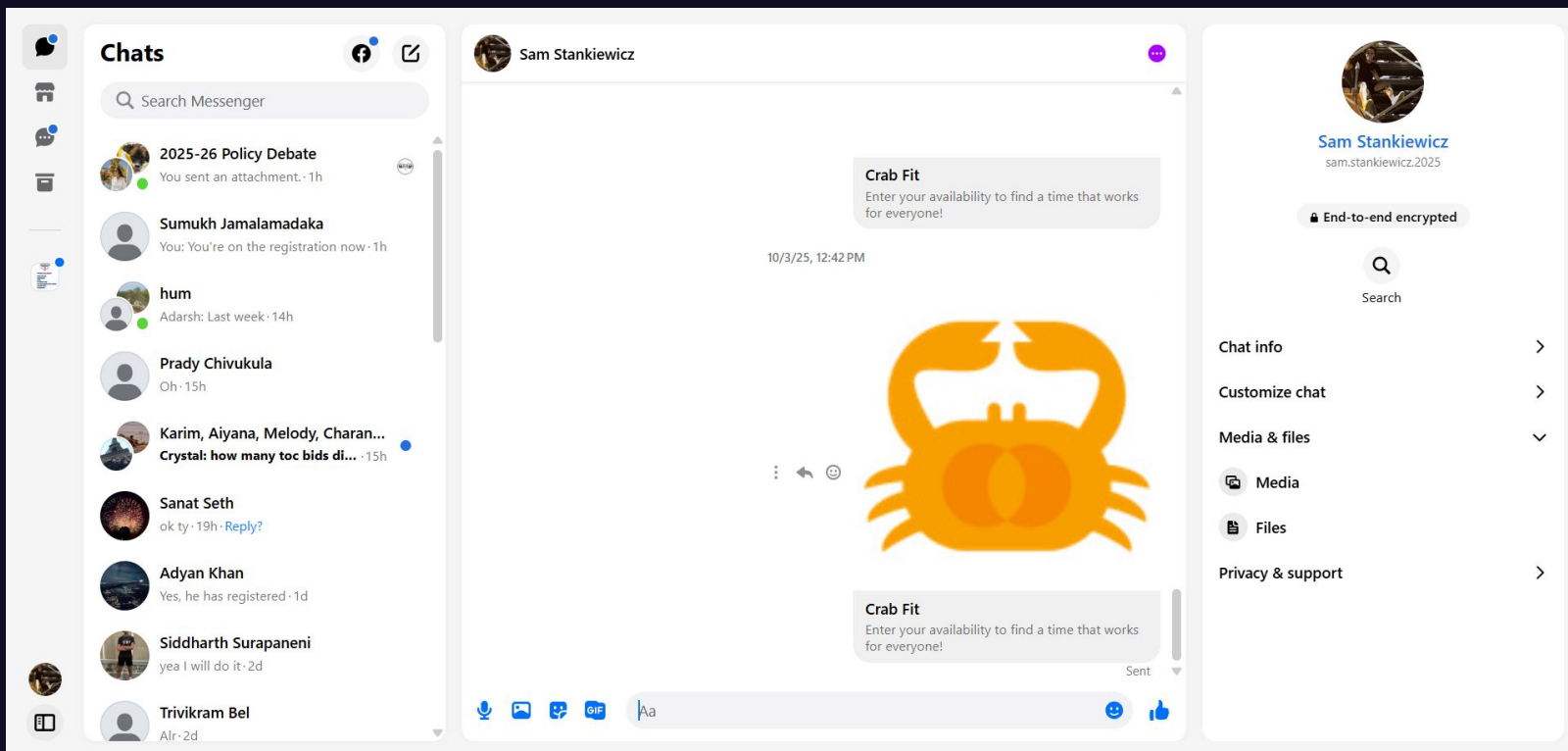


MESSENGER & GMAIL

The primary substitutes for text, Gmail and Messenger, are both poorly adapted for TJ-style communication

Messenger is blocked on school computers

Gmail is poorly adapted for messaging communication



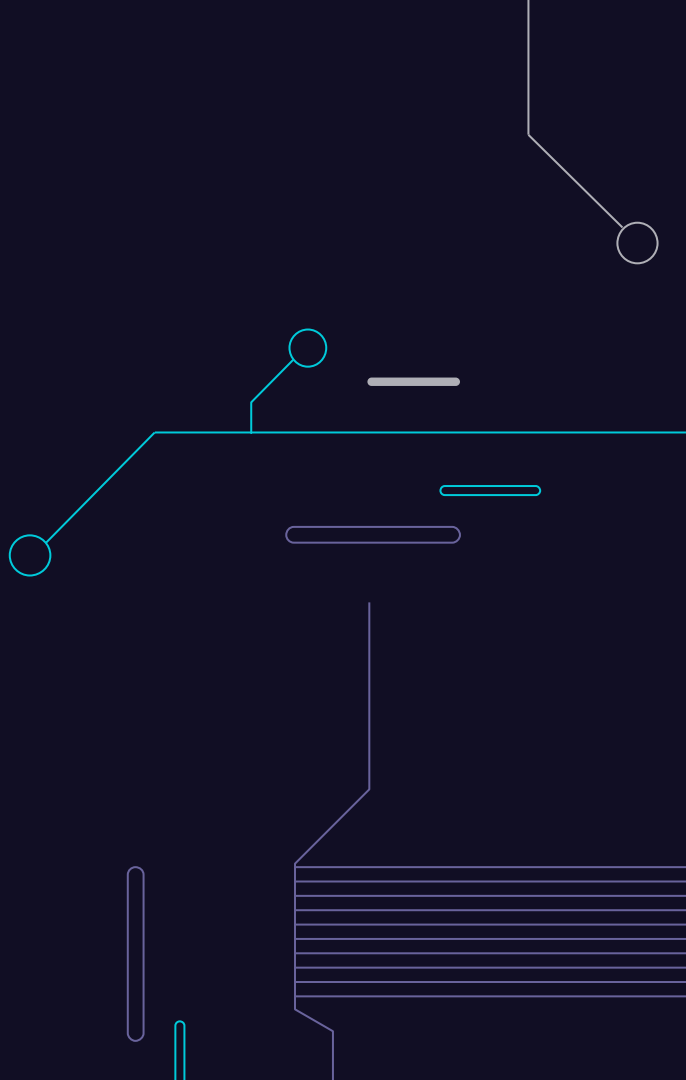
Messenger - Demo (10/6/2025)



iMessage - Demo (10/6/2025)

► Background

- Messaging App
 - Updating front end - matches data stored on back end
 - Encrypting occurs during this transfer of data between UI
 - Apps like messenger use a similar framework on a larger scale
- Ion OAuth
 - Allows for log in via Ion
 - Get, accurate, user data
- Will be hosted on Sam's personal domain (trulycalc.com)
 - TJ Director didn't end up working out



► Why JMessage?



Accessibility

Platform that works for a TJ student's workflow. Ion Directory streamlines most of searching for other students & logging in.



Networking

Ease of communication between clubs & club members, students in same classes, etc. - jump in communication capability.



CONTROLLED AUTH

Uses Ion integration to include a complete directory of all TJ students. Allows for secure communication with any TJ student that has used JMessage.



USE IN SCHOOL

Because our app is designed with a limited scope & function, it's ideal for use in school; it's also not banned on school devices.

► General Methodology

Database

We're using MongoDB with Redis for caching

- Document-based format for messaging system
- Better read/write efficiency at scale vs Firebase

Ion OAuth

Use pre existing OAuth code from sysadmins to integrate Ion accounts

- Limits to TJ students
- Improves signup/login efficiency

Framework

Frontend written in EJS with backend written in JS.

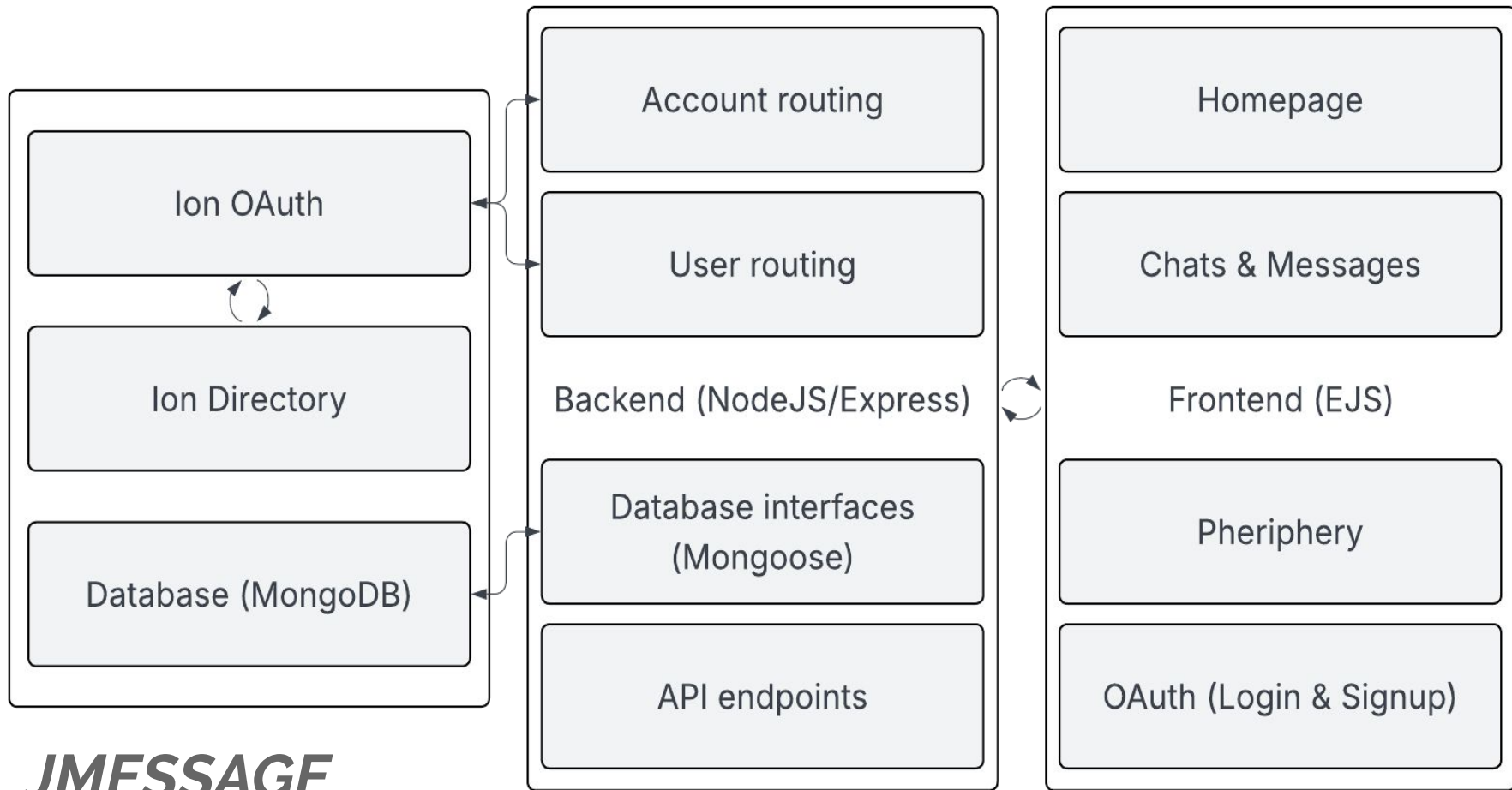
NodeJS/ExpressJS framework

- Faster than Flask
- Easier to build than NextJS

Webhooks

Integration of webhooks allows for instantaneous refresh on messages

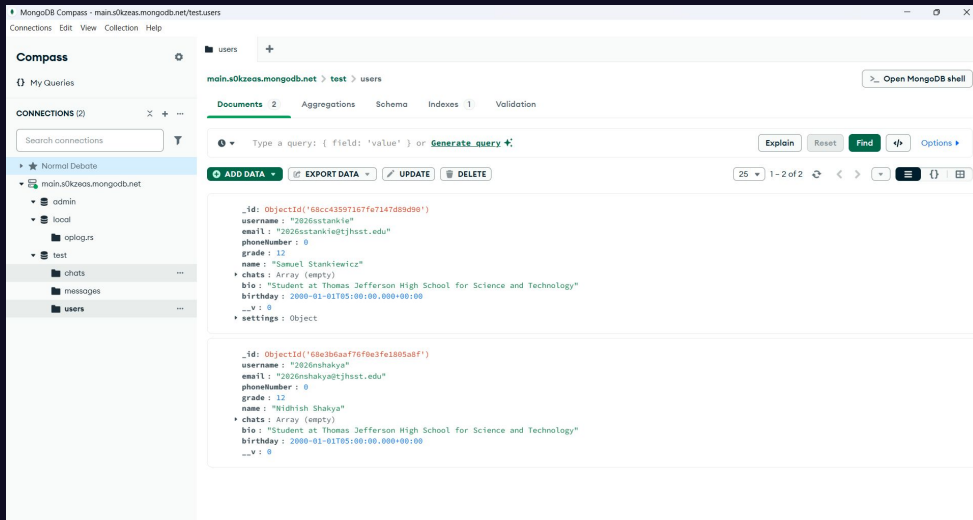
- Streamlines messaging process
- No need to constantly poll the server for new messages



JMESSAGE

Created by Samuel Stankiewicz with LucidChart

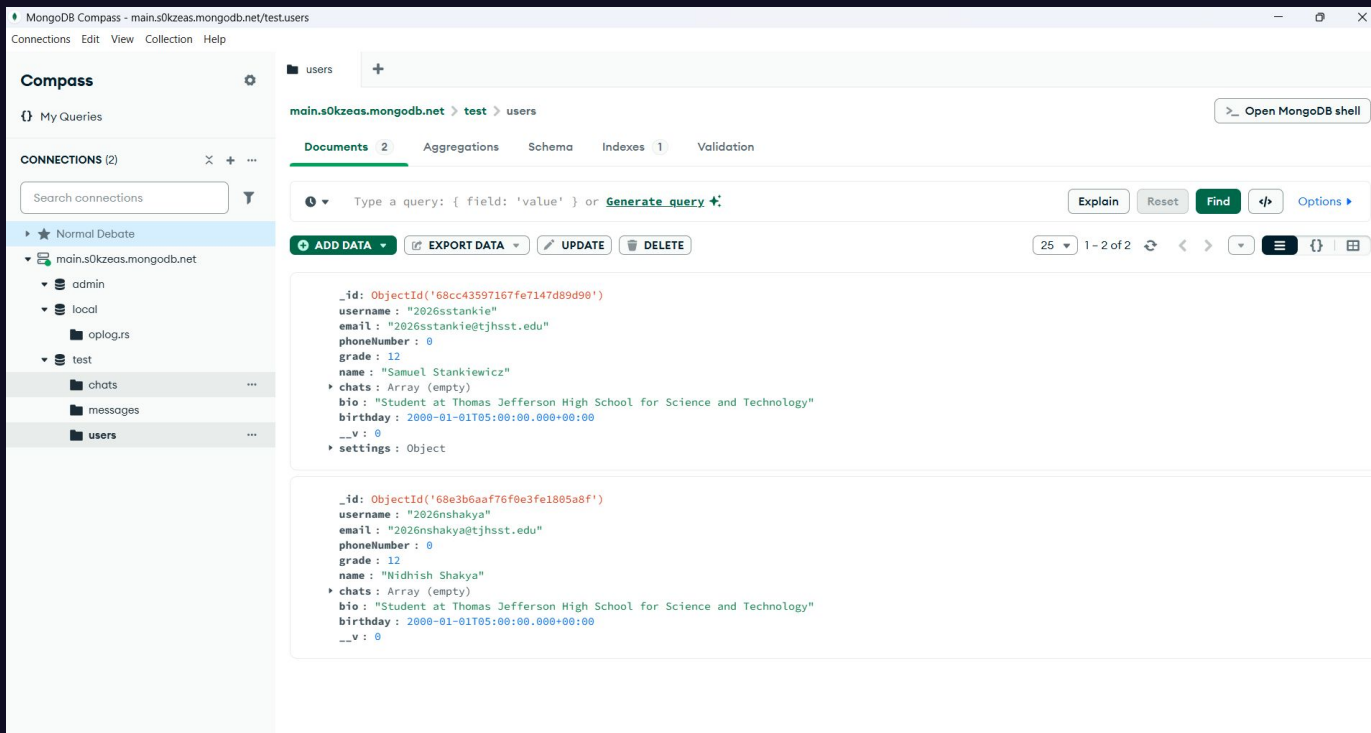
► Results - Login



Screenshot from MongoDB, Sam Stankiewicz

- Fully completed log in & storage of user information
 - Users stored as objects within MongoDB
 - Login information stored directly through ion
 - Ion OAuth verifies user upon login - we DON'T see your password

► Cont.



The screenshot shows the MongoDB Compass interface. On the left, the 'CONNECTIONS (2)' panel lists the 'main.s0kzeas.mongodb.net' connection, with the 'test' database selected. The 'users' collection is also visible. The main panel displays the 'users' collection with 2 documents. The first document is for Samuel Stankiewicz, and the second is for Nidhish Shakya. Both documents include fields for _id, username, email, phoneNumber, grade, name, chats, bio, birthday, __v, and settings.

main.s0kzeas.mongodb.net > test > users

Documents 2 Aggregations Schema Indexes 1 Validation

Type a query: { field: 'value' } or [Generate query](#)

ADD DATA EXPORT DATA UPDATE DELETE

25 1 ~ 2 of 2

```
{
  "_id": ObjectId("68cc43597167fe7147d89d98"),
  "username": "2026sstankie",
  "email": "2026sstankie@tjhsst.edu",
  "phoneNumber": 0,
  "grade": 12,
  "name": "Samuel Stankiewicz",
  "chats": Array (empty),
  "bio": "Student at Thomas Jefferson High School for Science and Technology",
  "birthday": 2000-01-01T05:00:00.000+00:00,
  "__v": 0,
  "settings": Object
}
```

```
{
  "_id": ObjectId("68e3b6aaf76f0e3fe1885a8f"),
  "username": "2026nshakya",
  "email": "2026nshakya@tjhsst.edu",
  "phoneNumber": 0,
  "grade": 12,
  "name": "Nidhish Shakya",
  "chats": Array (empty),
  "bio": "Student at Thomas Jefferson High School for Science and Technology",
  "birthday": 2000-01-01T05:00:00.000+00:00,
  "__v": 0
}
```

Screenshot from MongoDB, Sam Stankiewicz

► Results - Chats

```
_id: ObjectId('69de7080d9ef681c0fe34a46')
members: Array (2)
createdAt: 2026-04-14T16:51:12.856+00:00
media: Array (empty)
name: "test chat"
picture: null
chats: 0
read: Array (empty)
messageIds: Array (2)
__v: 0
```

Screenshot from MongoDB, Nidhish Shakya

- Chats = Objects in MongoDB
- Chats contain Ids for message “blocks”
 - Points towards chunks of messages that belongs to the specified chat
- When loading, server looks for all the chats the user is in
 - Messages are only loaded once the user opens a specific chat

► Results - Messages

```
_id: ObjectId('69ea2e808fd6334cc2a4901d')
fill: 19
▼ messages: Array (19)
  ▼ 0: Object
    sender: "2026nboukens"
    senderName: "Nick Boukens"
    text: "YO"
    timeSent: 2026-04-23T14:36:59.255+00:00
  ▶ 1: Object
  ▼ 2: Object
    sender: "2026nshakya"
    senderName: "Nidhish Shakya"
    text: "hey nick"
    timeSent: 2026-04-23T14:37:02.962+00:00
  ▶ 3: Object
  ▶ 4: Object
  ▶ 5: Object
```

Screenshot from MongoDB, Nidhish Shakya

- Messages = Object
 - Stores message that was sent
 - User who sent it
 - Date
- Messages are stored in “chunks” of 50
 - More space efficient, and makes loading a chat faster
 - Initially only load most recent chunk

► Cont.

```
_id: ObjectId('69ea3b333e936aa9a9017f5b')
fill: 2
▼ messages: Array (2)
  ▼ 0: Object
    sender: "2026nboukens"
    senderName: "Nick Boukens"
    text: "hello"
    timeSent: 2026-04-23T15:31:17.382+00:00
  ▼ 1: Object
    sender: "2026nshakya"
    senderName: "Nidhish Shakya"
    text: "hello"
    timeSent: 2026-04-23T15:31:21.873+00:00
index: 0
```

Screenshot from MongoDB, Nidhish Shakya

► Results - Searching

```
async function getNames () {
  try{
    const response = await fetch('/testSearch', {
      method: 'POST'
    });
    const result = await response.json();
    return result
  } catch (err) {
    console.log(err);
  }
  return 0
}
```

```
app.post('/testSearch', async (req, res) => {
  try{
    let aggUsers = [];
    await User.find({}, {name:1, _id:0})
      .then(users => {
        forEach(users, (user) => aggUsers.push(user['name']))
      });
    return res.send(aggUsers);
  } catch (err) {
    console.log(err);
  }
});
```

- Look through MongoDB for all templates that match a schema
 - In this case the schema will be the one we use for users
- Aggregate the names and ids found on that matched template
- Send it back to our javascript to update the html

► Results - Websockets

```
const httpServer = createServer(app);
const io = new Server(httpServer);

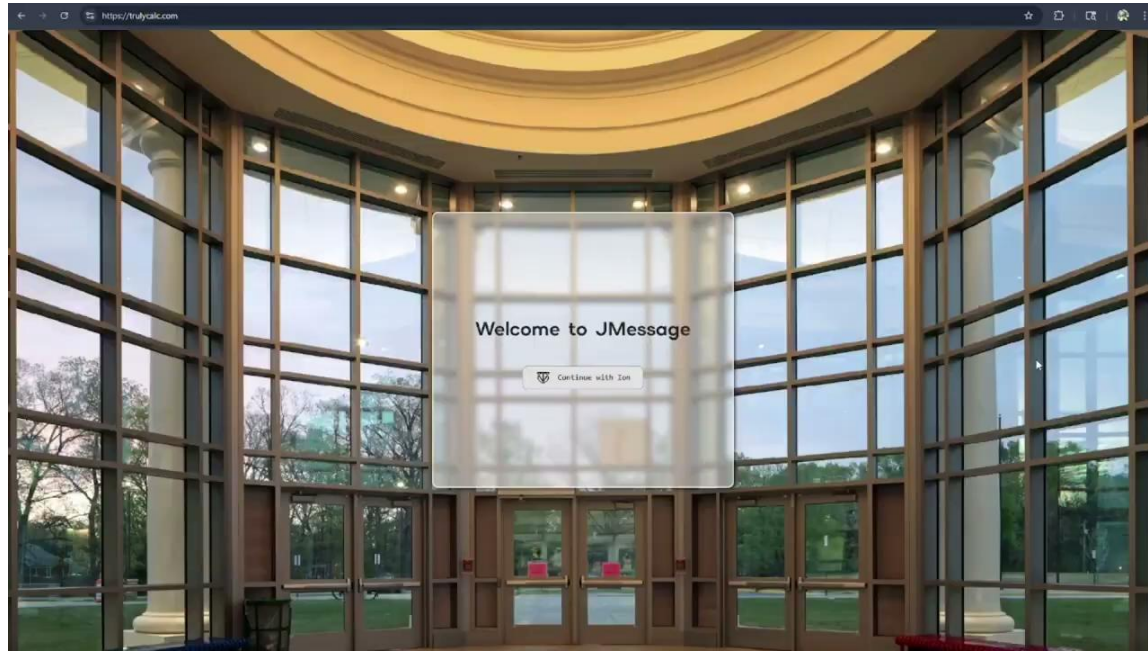
io.on('connection', (socket) => {
  socket.on('joinChat', (chatId) => {
    socket.join(chatId);
  });

  socket.on('leaveChat', (chatId) => {
    socket.leave(chatId);
  });
});
```

Screenshot from VSCode, Nidhish Shakya

- Creates a socket between the client and server
 - “Listens” for joining/leaving chats and sending/receiving messages
- Shoot back to the frontend to update webpage
- Seamless messaging, no need to constantly refresh to reload the newly added messages
- Avoids needless polling

DEMO



Recording from JMessage application, Nidhish Shakya. 5/14/2026.

► Limitations

Reliance on Ion

If Ion were to go down, then OAuth and JMessage, by relation, would also go down.

Time

While JMessage is fully functional as a messaging app, plenty of features, such as group chats, remain to be implemented fully.

Limited Usage

Since our app is based on Ion, there is no way for outside users to use it. Compatibility is also mostly focused on just computers & not phones.

Weaker Server

Currently our server isn't the strongest, high activity & usage would likely crash our the app.

CONCLUSIONS AND FUTURE WORK

Overall, JMessage is functional, it achieves our original “basic” ideas on what a lon-Based messaging app would look like and function life, however there are plenty of improvements to be made upon what we have. We’ve built the blueprint, with a majority of the backend/server-side functionality completed, but there’s still major room for improvement.

Crucial Additions

- Proper encryption (E2E) for messages & chats
- Better/More intuitive searching
- Upgrading server strength

Further Improvements

- Image compatibility and emojis
 - Sometimes functional, but can break on occasion
- Notification system/indicator

REFERENCES

[1] N. Kiran, "Securing Chat applications: Strategies for end-to-end encryption and cloud data protection," World Journal of Advanced Engineering Technology and Sciences, vol. 13, no. 2, pp. 528–540, Dec. 2024, doi:

<https://doi.org/10.30574/wjaets.2024.13.2.0634>.

[2] H.-Y. Wang, "A Review of Instant Messaging and Mobile Messaging Applications," International Journal of Economics, Finance and Management Sciences, vol. 7, no. 1, p. 13, 2019, doi:

<https://doi.org/10.11648/j.ijefm.20190701.13>.

[3] MongoDB, "MongoDB Documentation," <https://github.com/mongodb/docs-bi-connector/blob/DOCSP-3279/source/index.txt>, 2024.

<https://www.mongodb.com/docs/>

[4] MongoDB, "MongoDB Documentation," <https://github.com/mongodb/docs-bi-connector/blob/DOCSP-3279/source/index.txt>, 2024.

<https://www.mongodb.com/docs/>

[5] "Express 5.x - API Reference," expressjs.com.

<https://expressjs.com/en/5x/api.html>

Q&A

Any questions?

THANKS!